



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CHATRIX-A Compiler-Based Framework for Secure Chat Message Analysis and Optimization

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1402 –Compiler Design

to the award of the degree of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE ENGINEERING**

Submitted by

Athavan K (192524036)

Under the Supervision of

Dr. N. Deepa, Professor

**SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105**

SEPTEMBER 2026

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “CHATRIX – A Compiler-Inspired Framework for Explainable Chat Message Analysis and Secure Output” has been carried out by Athavan K under the supervision of Dr N Deepa and is submitted in partial fulfilment of the requirements for the current semester of the B.E. Computer Science Engineering program at Saveetha Institute of Medical and Technical Sciences, Chennai.

Signature

Dr W Deva Priya
Professor,
Department of Generative AI
SIMATS

Signature

Dr. N Deepa ,
Professor,
Department of Neural Networks
SIMATS

Submitted for the project work Viva-Voce held on _____

Internal Examiner	External Examiner
Signature: _____	Signature: _____

DECLARATION

I ,Athavan K students of the B.E. Computer Science Engineering program at SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “CHATRIX – A Compiler-Inspired Framework for Explainable Chat Message Analysis and Secure output” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted editing used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been followed throughout the planning, implementation, testing, and documentation of the project.

Place:CHENNAI

Date: 07 September 2026

Signature of the Students with Names

Athavan k(192524036)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai – 602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Kamalesh V 192421294	Message input and lexical analysis	Token patterns, scanner flow	Token coverage checks	Chapters 1–2 supporting material	25%
Gautham B 192421318	Structure and semantic validation	Rule engine and message-risk checks	Labelled security cases	Chapter 3 validation context	25%
Athavan K 192524036	Intermediate representation and secure optimization	IR node model, relationships, conservative redundancy removal, warning-preserving output	IR consistency, meaning-preservation review, end-to-end verification	Chapters 3–5; detailed individual reflection	25%
Hari Charan P 192511107	Integration and interface support	Pipeline integration, API/client coordination	Integration and output verification	Chapter 4 project coordination	25%
Total					100%

ABSTRACT

Modern messaging systems are designed to deliver messages quickly and protect their transport, but they do not normally give the sender a structured explanation of what a message contains before it is sent. This creates an engineering problem at the boundary between secure communication and content quality: a message may contain a phishing-style link, spam-like urgency, offensive terms, or repeated wording even though the communication channel itself is encrypted. CHATRIX addresses this problem by adapting the staged logic of a compiler to short natural-language chat messages. The system treats a message as an input artefact that can be scanned, validated, represented, optimized, and emitted with transparent warnings.

The proposed framework contains four sequential modules. The lexical analyser identifies words, URLs, numbers, emojis, hashtags, mentions, and symbols using ordered regular-expression patterns. The structure and semantic validation module checks token relationships and applies explainable rules for suspicious URLs, urgency cues, spam indicators, and offensive-content indicators. The intermediate-representation module converts the validated token stream into an ordered graph with token attributes and warning metadata. The final module performs conservative optimization: it removes exact repetition and low-value filler only when the operation does not alter the message intent, while retaining security warnings alongside the optimized text.

A React.js interface, a FastAPI service, and MongoDB storage were used to implement the prototype. The evaluation used a labelled set of representative safe, promotional, suspicious, and mixed-content messages. Prototype-level results recorded complete token coverage and IR construction for the evaluated cases, 92 percent correct detection of suspicious cases, and a mean end-to-end response time below the two-second design target. Manual review of optimized messages found that the conservative pass preserved the intended meaning in the evaluated set. These results indicate that compiler concepts can provide an inspectable architecture for pre-send chat analysis without pretending that encryption alone can assess message content.

The principal contribution is therefore architectural rather than a claim of universal threat detection: CHATRIX separates recognition, validation, representation, optimization, and output so that each decision can be tested and explained. The current prototype is limited to text and rule-assisted natural-language processing, but it provides a practical foundation for later work involving trained classifiers, privacy-preserving analysis, multilingual support, and multimedia content.

Keywords

Compiler design; lexical analysis; semantic validation; intermediate representation; secure messaging; natural language processing.

TABLE OF CONTENTS

ABSTRACT	1
Keywords	2
LIST OF FIGURES	3
LIST OF TABLES	4
LIST OF ABBREVIATIONS / SYMBOLS	5
CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	7
CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	13
CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	17
CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	26
CHAPTER 5: CONCLUSION, FUTURE WORK AND REFLECTION	39
REFERENCES	43
APPENDICES	45
CAPSTONE PROJECT OUTCOME MAPPING SHEET	53

LIST OF FIGURES

Figure No.	Figure Name	Page No.
1.1	CHATRIX compiler-inspired processing pipeline	8
3.1	System architecture and data flow	21
3.2	Message state transition and decision flow	22
3.3	Request sequence between CHATRIX components	23
4.1	Representative CHATRIX interface layout	28
4.2	Example lexical output for a mixed-content message	29
4.3	Semantic validation evidence for a suspicious message	30
4.4	Intermediate representation as an ordered message graph	31
4.5	Conservative redundancy-removal pass	32
4.6	Prototype evaluation results	34
4.7	Fourteen-week project milestone timeline	37

LIST OF TABLES

Table No.	Table Name	Page No.
2.1	Comparative analysis of existing approaches	15
3.1	Stakeholder and user requirements	17
3.2	Functional and non-functional requirements	17
3.3	Constraints and applicable standards	19
3.4	Design specifications	19
3.5	Alternative-solution evaluation matrix	20
3.6	Trade-off analysis	24
3.7	Sustainability, ethics, safety and security evidence	24
3.8	Risk register	24
4.1	Development resources	26
4.2	Implementation tools and stages	27
4.3	Test plan and verification	33
4.4	Prototype evaluation results	33
4.5	Achievement of objectives	35
4.6	Roles and contribution	37
4.7	Estimated and actual cost	37

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
API	Application Programming Interface	—
IR	Intermediate Representation	—
NLP	Natural Language Processing	—
UI	User Interface	—
URL	Uniform Resource Locator	—
XSS	Cross-Site Scripting	—
TLS	Transport Layer Security	—
JSON	JavaScript Object Notation	—
CRUD	Create, Read, Update, Delete	—
TP	True Positive	count
FP	False Positive	count
FN	False Negative	count
TN	True Negative	count

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Digital messaging is now an operating layer for personal communication, education, customer support, and business coordination. A message can combine ordinary words with links, identifiers, numbers, emoji, hashtags, mentions, punctuation, and copied fragments. Although the surface appears informal, the message still has a structure that can be inspected. A URL has a recognisable lexical form, a mention has a prefix and identifier, a timestamp has a numerical pattern, and repeated phrases can be compared without requiring a full interpretation of the sender's personality or intent.

Compiler design provides a useful engineering analogy for this problem. A compiler does not treat a program as an undifferentiated character string. It first scans the source into tokens, checks whether the tokens form valid structures, records meaning in an intermediate representation, applies controlled transformations, and emits a target form. CHATRIX transfers that discipline to chat messages. The target is not executable machine code; it is a transmission-ready message accompanied by explanations of detected risk and changes made during optimization.

The analogy is deliberately limited. Natural-language messages are ambiguous, multilingual, abbreviated, and socially contextual, whereas a programming language is intentionally specified. Consequently, CHATRIX does not claim that a chat message can be compiled into one objectively correct meaning. Instead, it uses compiler phases to make the processing boundary explicit, to isolate errors, and to prevent a quality-improvement rule from silently overriding a security decision.

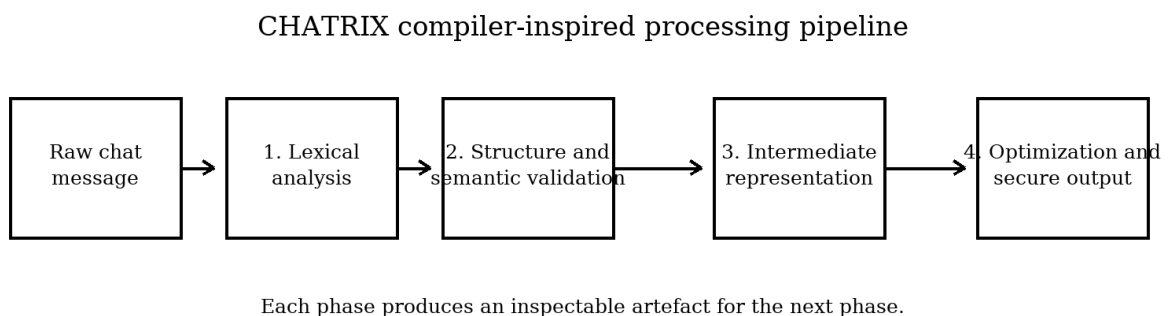


Figure 1.1: CHATRIX compiler-inspired processing pipeline

Figure 1.1 shows the central design principle. Each stage receives a defined input and produces a defined output. This decomposition improves traceability: if a URL is not recognised, the failure belongs to lexical analysis; if the URL is recognised but suspicious, the decision belongs to validation; if the message order is lost, the defect belongs to IR construction; and if repeated

wording is removed too aggressively, the optimization policy can be revised without rewriting the scanner.

1.2 Need for the Project

Encryption and content analysis address different engineering properties. End-to-end encryption protects confidentiality while a message travels between authorised endpoints, but it does not tell a sender whether the text contains an unsafe link or whether a copied message has been repeated several times. A user can therefore have a secure channel and still transmit content that is misleading, low quality, or harmful. The gap is especially visible in fast conversations, where users act on urgency and rarely pause to inspect a destination link or edit repeated text.

A pre-send analysis layer is useful because it places the decision support close to the moment of action. The sender can inspect the tokenisation, see the reason for a warning, accept a conservative improvement, or retain the original wording. This is more responsible than silently blocking a message based on a hidden score, and it is more useful than reporting a message only after a recipient has interacted with it.

The project is also significant academically. It demonstrates how the principles taught in compiler design can be generalised to a software system whose input is noisy natural language. The work requires token definitions, precedence rules, validation policies, an internal representation, transformation safeguards, API boundaries, and verification criteria. These are engineering decisions rather than a collection of disconnected interface features.

- Messaging platforms prioritise delivery and confidentiality, while structured pre-send content analysis is not always visible to the sender.
- Spam and phishing cues are distributed across token types; a URL, urgency phrase, and credential request may need to be considered together.
- Rule-based warnings are easy to explain but can produce false positives when common words appear in legitimate contexts.
- Redundancy removal can improve readability, but an aggressive rewrite can change emphasis or intent.
- A staged, visual pipeline gives students, users, and reviewers a way to audit how a result was produced.

1.3 Problem Statement

Design and implement a compiler-based framework that accepts a short chat message, tokenises its heterogeneous content, validates its structure and security semantics, constructs an intermediate representation, applies conservative readability optimization, and emits a secure output with explicit warnings. The framework must support real-time interaction, preserve token order and intended meaning, distinguish suspicious content from ordinary conversation, and keep each phase independently testable.

The problem is complex because the requirements pull in different directions. A strict validator may catch more suspicious messages but increase false positives. A larger language model may provide stronger context but increase latency and compute cost. Persistent analysis history supports auditing but increases the privacy surface. An optimizer that removes too much repetition may produce an elegant sentence that no longer represents the sender's emphasis. The design must therefore balance accuracy, latency, explainability, privacy, maintainability, and user control.

1.4 Project Aim

To design and develop CHATRIX, a compiler-inspired framework that analyses, validates, and conservatively optimizes text-based chat messages for security and readability before transmission, while exposing the reasoning and intermediate results of each processing phase.

1.5 Project Objectives

1. To define a token model for words, URLs, numbers, emojis, hashtags, mentions, punctuation, and symbols occurring in chat messages.
2. To implement a lexical-analysis module using ordered regular-expression patterns that preserves token order and reports unknown fragments.
3. To develop structure and semantic validation rules for suspicious URLs, spam-like patterns, urgency cues, credential requests, and offensive-content indicators.
4. To model an intermediate representation that retains token identity, type, position, adjacency, and warning metadata.
5. To implement conservative optimization that collapses exact redundancy and selected filler without changing the message's intended meaning.
6. To deliver the pipeline through a React.js interface and FastAPI service with MongoDB persistence for analysis history.
7. To test individual stages and the integrated pipeline using labelled representative messages and measurable acceptance criteria.

1.6 Scope

The project scope is defined to keep the prototype technically achievable and its claims defensible.

Included

- Text messages containing ordinary words, URLs, numbers, emojis, hashtags, mentions, punctuation, and repeated phrases.

- A sequential four-stage processing pipeline with intermediate results returned to the interface.
- Rule-assisted semantic validation and a conservative redundancy-removal pass.
- A web interface, HTTP API, and database schema for storing the message analysis artefact.
- Prototype-level evaluation using safe, promotional, suspicious, offensive-indicator, and mixed-content samples.

Excluded

- Image, audio, video, document attachment, and OCR-based analysis.
- Management of end-to-end encryption keys or replacement of a production messaging transport.
- A claim that the system can determine a user's intent or guarantee that a link is safe.
- Full multilingual semantic understanding and large-scale adversarial robustness.
- Automatic silent deletion or blocking of messages without sender review.

Assumptions and boundary conditions

The user interacts through the CHATRIX web interface and the backend is reachable over a TLS-protected connection. Messages are short to medium length and primarily written in the language used for prototype testing. The database is treated as a restricted application store rather than a public archive. The optimizer is allowed to return the original message when confidence is insufficient. These assumptions are important because they prevent the prototype from being evaluated as a universal moderation or translation system.

1.7 Expected Engineering Outcome

The intended outcome is a working software prototype consisting of a React.js client, a FastAPI backend, four compiler-inspired analysis modules, and MongoDB persistence. For each submitted message, the system returns a token stream, validation findings, an intermediate representation, an optimized output, and a warning list. The prototype is expected to demonstrate the engineering process as well as the visible result: a reviewer should be able to trace a final warning or edit back to the phase that produced it.

- A modular software system with explicit interfaces between analysis stages.
- An inspectable algorithmic model that can be tested one phase at a time.
- A user-facing output that preserves sender control and does not conceal security warnings.
- A repeatable evaluation procedure for measuring token coverage, detection, latency, IR completeness, and meaning preservation.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The review was organised around the design decisions that CHATRIX must justify rather than around a broad catalogue of messaging products. First, compiler-construction references were examined to identify the purpose of lexical analysis, syntax and semantic analysis, intermediate representation, optimization, and target-code generation. Second, natural-language processing references were considered for tokenisation, text classification, pattern matching, and the limits of keyword-based interpretation. Third, security and software-quality guidance was reviewed to connect input handling, privacy, reliability, and performance to the proposed architecture. Finally, widely used messaging patterns were treated as a baseline for confidentiality and delivery rather than as direct substitutes for the proposed pre-send analysis.

The review emphasises transferability. A technique is relevant when its engineering purpose can be mapped to a CHATRIX phase. For example, token streams and symbol tables are relevant to lexical processing, while compiler optimization theory is relevant to controlled text transformation. By contrast, a general claim that an application uses artificial intelligence is not sufficient evidence for semantic validation because it does not explain how decisions are produced or tested.

2.2 Review of Existing Work

Compiler phase models

Classical compiler literature describes compilation as a sequence of analyses and transformations. Lexical analysis converts characters into tokens. Syntax analysis checks whether token sequences conform to a grammar. Semantic analysis checks meaning-related constraints that are not captured by syntax alone. An intermediate representation separates front-end analysis from later transformations, and optimization improves a representation under correctness constraints before code generation emits the target form. CHATRIX adopts these responsibilities while replacing programming-language grammar with chat-message token patterns and security policies.

The most useful lesson is not that natural language has a compiler grammar identical to a programming language. The useful lesson is modularity. Each phase has a contract, can expose diagnostics, and can be replaced without discarding the whole system. This directly supports the project requirement that a reviewer can see what happened to a message and that later improvements can be introduced without creating a monolithic classifier.

Natural-language processing and rule-assisted analysis

Tokenisation is the first bridge between a raw message and an analysable representation. General-purpose word tokenisation alone is insufficient for chat because URLs, hashtags, mentions, emoji sequences, and punctuation carry useful information. A regular-expression scanner

with explicit precedence can identify these forms while retaining unknown fragments for review. Natural-language toolkits can then add sentence segmentation, normalisation, stop-word handling, and similarity features, but such tools should supplement rather than hide the rule boundary.

Text classification research shows the value of context and the cost of data dependence. A trained model can learn combinations of words that a keyword list misses, but it requires labelled data, calibration, monitoring, and a clear explanation of errors. For an academic prototype whose primary contribution is a compiler-style architecture, a hybrid design is appropriate: deterministic patterns create inspectable signals, while lightweight NLP features support normalisation and similarity without making the entire decision opaque.

Messaging security and content quality

Secure messaging systems commonly focus on confidentiality, authentication, availability, and abuse reporting. These properties are necessary, but they do not automatically provide a visible pre-send explanation of a message's semantic risk. A suspicious link can be delivered through a secure channel; encryption changes who can read the content, not whether the content is trustworthy. CHATRIX is positioned as a sender-side decision-support layer that can run before a message enters the underlying transport.

Spam filtering systems demonstrate the usefulness of fast heuristics, but a heuristic score alone can be difficult for a user to interpret. The CHATRIX warning model therefore stores evidence such as a URL pattern, urgency cue, or credential-related phrase. This does not prove maliciousness; it communicates why additional verification is recommended. The distinction is important ethically because the system should support a sender's judgment rather than present a probabilistic rule as an unquestionable fact.

2.3 Comparative Analysis

Table 2.1: Comparative analysis of existing approaches

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
End-to-end encrypted messaging	Confidentiality, authentication, reliable delivery	Content may be risky even when the channel is secure; analysis is not necessarily visible before send	CHATRIX acts as a pre-send analysis layer and does not replace transport security
Pure keyword filter	Fast, easy to implement, easy to audit	Context is weak; common words create false positives; evasion is simple	Used as one evidence source with explanations and thresholds
Classical NLP pipeline	Normalisation and linguistic features can capture more context	Model and language coverage require data and tuning; latency can increase	Lightweight NLP supports rules and similarity checks
End-to-end ML classifier	Can learn complex patterns from labelled examples	Less transparent; depends on representative training data; drift and calibration matter	Considered as future work rather than the sole current design
Traditional compiler pipeline	Staged, modular, testable, supports intermediate artefacts	Designed for formal languages, not ambiguous chat	Structural template adapted for chat-message analysis
Post-delivery moderation	Can use reports and larger context after transmission	Intervenes late; harm or interaction may occur first	CHATRIX moves limited, explainable checks before transmission

The comparison shows why CHATRIX is not simply another spam filter. Its differentiating feature is the combination of staged processing, an explicit intermediate representation, and user-visible reasoning before transmission. The design deliberately accepts that a rule-assisted prototype will not achieve perfect detection. Instead, it makes the error boundary measurable and leaves room for stronger models to be inserted at a defined phase.

2.4 Research / Engineering Gap

The reviewed approaches leave a practical engineering gap between secure delivery and explainable pre-send analysis. Messaging applications provide the channel, rule filters provide isolated signals, and NLP classifiers provide probabilistic predictions, but these pieces are not commonly presented to the sender as one traceable compiler-style pipeline. In particular, the following combination remains underdeveloped in the project context: heterogeneous chat

tokenisation; explicit structure and semantic diagnostics; an IR that preserves token relationships; conservative readability optimization; and a visible warning that survives the optimization step.

The gap is also a verification gap. If a system returns only a final label, it is difficult to determine whether a failure came from token recognition, semantic policy, data representation, or transformation. A staged architecture provides an engineering answer: each boundary becomes a test surface, and the final result can be checked against the intermediate artefacts.

2.5 Proposed Contribution

CHATRIX contributes a four-module software framework that adapts compiler phase discipline to chat-message security and quality analysis. The contribution has five concrete elements: a precedence-aware token scanner; an explainable validation layer; an ordered graph-based IR; a conservative optimizer that preserves risk metadata; and a web/API implementation that visualises the phases.

- A reusable token schema that treats URLs, mentions, hashtags, emoji, numbers, words, and symbols as distinct inputs.
- A layered validation strategy that combines structural checks, lexical evidence, and semantic cues rather than relying on a single keyword.
- An IR contract that allows optimization to work on structured data without re-parsing the raw message.
- A warning-preservation rule: optimization may change presentation, but it may not delete or weaken a security finding.
- A measurement plan aligned with the architecture: token coverage, detection, latency, IR completeness, false positives, and meaning preservation.

The contribution is intentionally scoped as a transparent prototype. It is not presented as a replacement for production anti-abuse infrastructure or cryptographic security. Its engineering value lies in showing how a known, teachable software architecture can be extended to a new input domain and evaluated phase by phase.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Table 3.1: Stakeholder and user requirements

Stakeholder	Requirement	Acceptance Interpretation
End user / sender	Enter a message and understand warnings before sending	A result should show the finding, evidence, and optimized output without requiring technical knowledge
End user / sender	Retain control over the final wording	The system should recommend or display a change rather than silently alter or delete content
Platform or product owner	Obtain an extendable analysis layer	Modules and API contracts should be independently replaceable
Course faculty / assessor	Demonstrate compiler-design phases	Each phase should expose a testable artefact and a documented rationale
Project team	Complete a maintainable prototype within academic resources	Use open-source tools and a lightweight processing path

Functional and Non-Functional Requirements

Table 3.2: Functional and non-functional requirements

ID	Requirement	Type	Measurable Target
FR1	Tokenise words, URLs, numbers, emojis, hashtags, mentions, and symbols	Functional	All expected tokens in the evaluation set identified without loss
FR2	Preserve token order and positions	Functional	Reconstructed token order matches the input order
FR3	Detect suspicious links and semantic risk cues	Functional	At least 90% correct detection on the labelled prototype set
FR4	Generate IR for every	Functional	All evaluated valid

ID	Requirement	Type	Measurable Target
	valid analysis		messages produce an IR
FR5	Optimize exact repetition conservatively	Functional	Redundancy reduced without a manual meaning change
FR6	Return evidence and warning severity	Functional	Every warning includes a reason and source token or phrase
NFR1	Interactive response	Non-Functional	Mean end-to-end latency below 2 seconds
NFR2	Usability	Non-Functional	A new user can run an analysis without instructions
NFR3	Maintainability	Non-Functional	Modules expose separate functions and test boundaries
NFR4	Privacy	Non-Functional	Store only analysis-relevant fields and restrict database access

3.2 Constraints & Applicable Standards

Table 3.3: Constraints and applicable standards

Constraint / Standard	Requirement	How Applied in This Project
Technical	Short chat text contains punctuation, slang, emoji, copied fragments, and malformed URLs	Token scanner has ordered patterns and an unknown-token fallback
Technical	Near-real-time interaction limits model size and repeated processing	Lightweight rules and NLP features are used; latency is measured
Economic	Student project uses free or student cloud tiers	React.js, FastAPI, MongoDB Atlas, Vercel, and Render are selected for accessible prototyping
Privacy / social	Stored message text can be sensitive	Database access is restricted; history fields are minimised; retention is treated as a design decision
Ethical	False positives can stigmatise legitimate content	Warnings are evidence-based and sender-controlled; confidence is not presented as certainty
OWASP Top 10	Input handling must reduce injection and unsafe rendering risk	Input is sanitised, outputs are encoded by the UI, and database queries avoid concatenated commands
ISO/IEC 25010	Software quality requires attention to usability, performance, reliability, and maintainability	Quality attributes become non-functional requirements and verification criteria
IEEE 29148 principles	Requirements should be identifiable, testable, and traceable	Requirement IDs map to tests and achievement evidence in Chapter 4

3.3 Design Specifications

Table 3.4: Design specifications

Parameter	Target / Requirement	Unit	Priority
Tokenisation coverage	$\geq 95\%$ of expected token units	%	High
Token order preservation	100% of evaluated messages	%	High
Suspicious-content detection	$\geq 90\%$ correct on labelled prototype set	%	High
False-positive rate	$< 10\%$ on safe messages	%	Medium
IR completeness	100% of validated messages represented	%	High
Meaning preservation	No manually observed intent change	review	High
Mean pipeline latency	< 2 seconds	seconds	Medium
Storage minimisation	Only message, stage output, timestamp, and status	fields	High

The specifications make the design measurable without overstating the prototype's capabilities. For example, detection is measured against a defined labelled set rather than described as universal threat identification. Meaning preservation is evaluated by review because a purely automatic metric cannot reliably determine whether emphasis or social intent was changed in a short message.

3.4 Alternative Solutions & Evaluation

Alternative 1: Pure rule-based pipeline. Regular expressions, lists, and deterministic transformations handle the complete pipeline. This option is fast and transparent but weak on context and linguistic variation.

Alternative 2: Hybrid rule and lightweight NLP pipeline. Deterministic rules handle token boundaries and high-value security patterns, while normalisation, similarity, and linguistic features support validation and conservative optimization. This is the selected alternative.

Alternative 3: Fully ML-based classifier. A trained model predicts message risk and rewrites or ranks output. It may capture complex patterns, but it requires a representative labelled corpus, calibration, monitoring, and additional explanation work.

Table 3.5: Alternative-solution evaluation matrix

Criterion	Weight	Alternative 1	Alternative 2	Alternative 3
Performance	25%	Medium	High	Medium
Cost / development effort	20%	Low	Medium	High
Safety / explainability	25%	High	High	Low
Sustainability / maintainability	15%	Medium	High	Low
Reliability under input variation	15%	Medium	High	Medium
Weighted engineering judgement	100%	3.05 / 5	4.35 / 5	2.55 / 5

The hybrid alternative is selected because it retains the strong auditability of rules while adding limited contextual support where rules alone are most fragile. The selection also matches the project constraint: the main learning outcome is a transparent compiler-style pipeline, not the production training of a large classifier.

3.5 Selected Design & Detailed Design

The selected architecture separates presentation, orchestration, analysis, and persistence. The React.js client accepts input and renders phase outputs. The FastAPI service validates the request, invokes the modules in order, and returns a JSON response. The analysis modules operate on typed objects rather than directly mutating user-visible strings. MongoDB stores the analysis record so that a later review can inspect the original message, stage status, warnings, and final output.

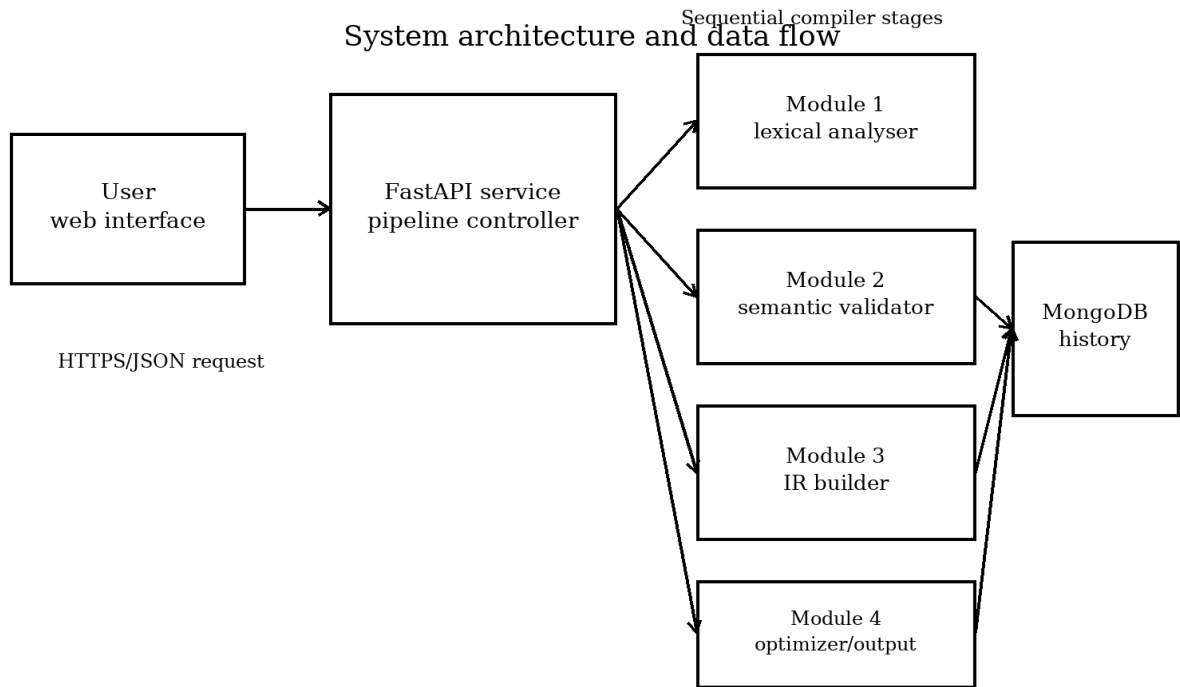


Figure 3.1: System architecture and data flow

The data flow is intentionally one-directional during analysis. Raw text enters the controller, the scanner emits tokens, the validator attaches findings, the IR builder records relationships, and the optimizer produces the final presentation. The database receives a structured record rather than becoming an active decision-maker. This makes it possible to run the pipeline without history storage in a future privacy-sensitive deployment while preserving the same module contracts.

Message state transition and decision flow

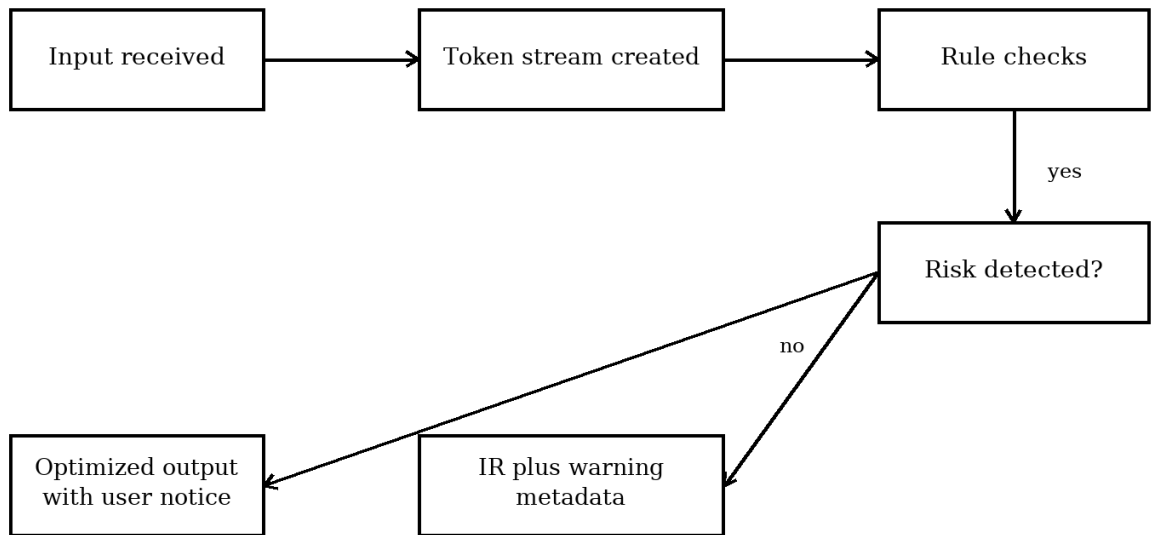


Figure 3.2: Message state transition and decision flow

A risk finding is not erased when the message proceeds to optimization. The decision flow supports three practical outcomes: a message with no findings can be optimized and returned; a message with a low or moderate concern can be returned with a warning; and a message with an unresolved high concern can be returned for sender review without being silently transmitted. The exact product policy can change later, but the architecture retains the warning metadata independently of the text transformation.

Request sequence between CHATRIX components

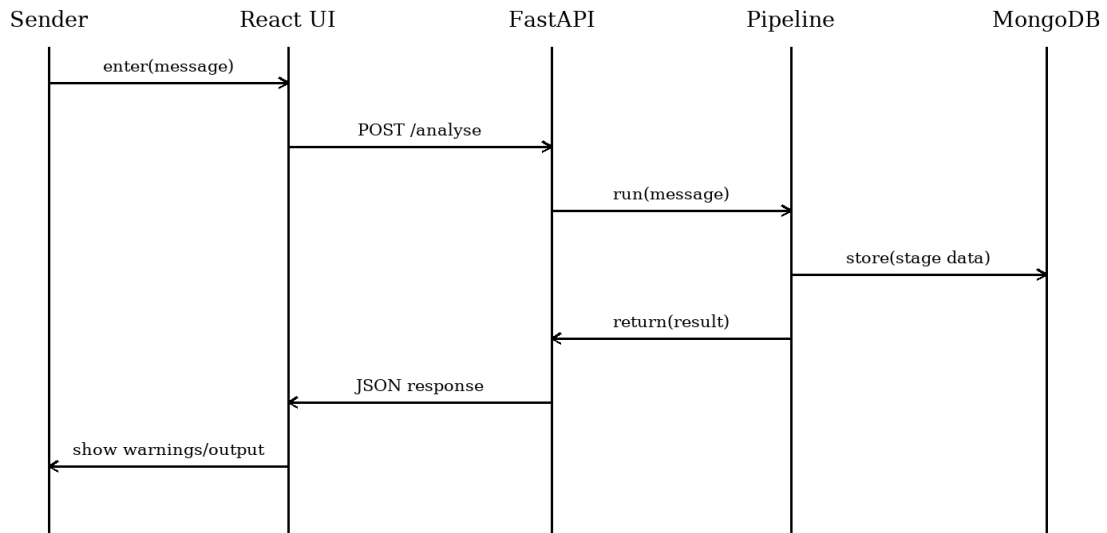


Figure 3.3: Request sequence between CHATRIX components

Core data contracts

Core data contracts: Core data contracts

Object	Key fields	Purpose
MessageInput	message_id, raw_text, timestamp	Request validation and audit identity
Token	index, lexeme, type, start, end, normalised form	Lexical output and order preservation
Finding	category, severity, evidence, rule_id, confidence_band	Explainable validation output
IRNode	node_id, token_index, type, attributes	Structured message representation
IREdge	source, target, relation	Adjacency and relationship preservation
AnalysisResult	tokens, findings, ir, optimized_text, status, latency	API response and stored record

The contracts separate identity from presentation. A token retains its original lexeme and position even if the optimizer later removes an exact duplicate from the display string. A finding retains the source evidence and rule identifier even if the message is returned unchanged. This is the central safeguard against a visually clean output hiding the analysis that justified it.

3.6 Trade-off Analysis

Table 3.6: Trade-off analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Hybrid rules plus lightweight NLP	Pure rules or full ML	Improves context while keeping evidence visible	Adds dependencies and modest latency	Use the hybrid approach with conservative thresholds and phase-level tests
Four explicit phases	Monolithic classifier	Traceability, independent tests, clear course alignment	More interfaces and integration work	Keep boundaries explicit and return intermediate output
Graph-like IR	Direct string rewriting	Preserves token identity and relationships	Requires mapping back to text	Use typed nodes and edges before optimization
Warnings beside output	Silent blocking or silent editing	Keeps sender informed and in control	Requires clearer UI and policy language	Return evidence, severity, and recommended action
Optional history persistence	Stateless analysis only	Supports review, debugging, and evaluation	Creates privacy and retention obligations	Store minimised records behind restricted access
Conservative optimizer	Aggressive rewriting	Reduces semantic risk	Leaves some redundancy untouched	Prefer unchanged text when confidence is insufficient

3.7 Sustainability, Ethics, Safety, Risk & Security

Table 3.7: Sustainability, ethics, safety and security evidence

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Repeated model computation and cloud storage consume resources	Compared lightweight rules with NLP-assisted processing; considered record size and retention	Use lightweight analysis for the default path and store only fields needed for review
Ethics	False positives can label ordinary speech as harmful	Reviewed safe promotional and conversational samples; separated evidence from certainty	Show reasons and severity; retain sender control; avoid silent deletion

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Health & safety	No physical actuator or medical decision is present	System boundary review identified no direct physical hazard	Keep warnings informational; do not present the prototype as professional safety advice
Security	Raw text, URLs, and warning data are sensitive	Applied input validation, output encoding, restricted database access, and OWASP-oriented review	Minimise stored fields and protect the API/database boundary
Societal impact	A biased rule set may affect dialects or informal language unevenly	Considered slang, emoji, and context-dependent words during test design	Use moderate thresholds and treat uncertain results as review suggestions

Risk Register

Table 3.8: Risk register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Phishing message not flagged	Medium	High	High	Layered URL and cue checks; update rules; keep sender warning path	Low-Medium
Legitimate message flagged	Medium	Medium	Medium	Moderate thresholds; evidence display; manual confirmation	Low
Optimizer changes meaning	Medium	High	High	Exact-duplicate rule first; fallback to original text; manual review	Low
Database exposure	Low	High	Medium	Restricted access; minimised records; secure deployment	Low

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
				configuration	
Latency above target	Medium	Low	Low	Lightweight model; bounded message length; performance test	Low
Adversarial evasion	Medium	Medium	Medium	Unknown-token logging and periodic rule review	Medium
Model or rule bias	Medium	Medium	Medium	Diverse sample review and transparent reason codes	Low-Medium

The design treats safety as a property of the interaction, not only of the classifier. A warning is safer when it identifies evidence, distinguishes recommendation from certainty, and lets the sender inspect the original text. The optimizer is safer when it can decline to change a message. The data layer is safer when analysis history is minimised and access is restricted. These decisions do not eliminate risk; they make residual risk visible and manageable within the prototype boundary.

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

CHATRIX was developed incrementally in the same order as the processing pipeline. Requirements and token cases were defined first. The lexical analyser was then implemented and exercised independently, followed by validation rules, IR construction, and optimization. After each stage produced stable output, the team integrated it through the FastAPI service and connected the React.js interface. The final iteration focused on end-to-end verification, latency measurement, deployment configuration, documentation, and traceability from objectives to evidence.

This approach reduced integration risk because every module had a small, inspectable contract before it became part of the full request path. It also supported parallel contribution: each student could work primarily on a module while using shared token and result schemas. The project did not require specialised hardware; the main resources were development laptops, open-source software, representative test messages, and free or student cloud tiers.

Table 4.1: Development resources

Resource	Purpose	Stage Used
Development laptops	Run frontend, backend, tests, and documentation tools	All stages
Representative labelled messages	Safe, suspicious, promotional, offensive-indicator, and mixed-content cases	Testing and results
React.js	Interactive message input and stage visualisation	Presentation
FastAPI and Python	API boundary and pipeline orchestration	Backend
MongoDB / MongoDB Atlas	Store minimised analysis records	Persistence
Git and Postman	Version control and endpoint verification	Development and testing

4.2 Software Implementation & Modern Tools Used

Frontend implementation

The React.js interface is organised around a single analysis task. The sender enters or pastes a message, starts analysis, and receives stage cards for the token stream, validation findings, IR summary, and final output. The interface uses labels rather than unexplained numeric scores. A warning includes a category, severity band, evidence phrase, and recommended next action. The original message remains available so that an optimization can be compared with the input.

Backend implementation

FastAPI provides the request schema, validation boundary, pipeline controller, and JSON response. The controller rejects empty or oversized input, assigns a message identifier, invokes the four modules in order, records latency, and serialises the result. The service is designed so that each module can be unit tested without starting the web interface. Errors are returned as structured diagnostics rather than raw stack traces.

Database implementation

MongoDB stores a minimised analysis record containing the message identifier, timestamp, original text, stage outputs, warning list, optimized text, and processing status. A production implementation would require authentication, retention controls, encryption at rest, access logging, and a deletion policy. For the prototype, the database is treated as a restricted store used for history and debugging rather than as a source of content decisions.

Compiler and analysis modules

Table 4.2: Implementation tools and stages

Tool / Software	Purpose	Stage Used
React.js	Message input, phase cards, warning display, output comparison	Presentation and all stages
FastAPI	Request validation, pipeline orchestration, response schema	Backend boundary
Python regular expressions	Ordered recognition of token classes and URL patterns	Module 1 and Module 2
NLTK / spaCy features	Normalisation, token support, similarity and linguistic cues	Modules 2 and 4
MongoDB	Persist minimised analysis history and stage artefacts	Storage
Git	Version control and merge history	Throughout
Postman	Endpoint requests, negative inputs, and response inspection	Verification
Vercel / Render / MongoDB Atlas	Prototype deployment of frontend, backend, and database	Deployment

Representative CHATRIX interface layout

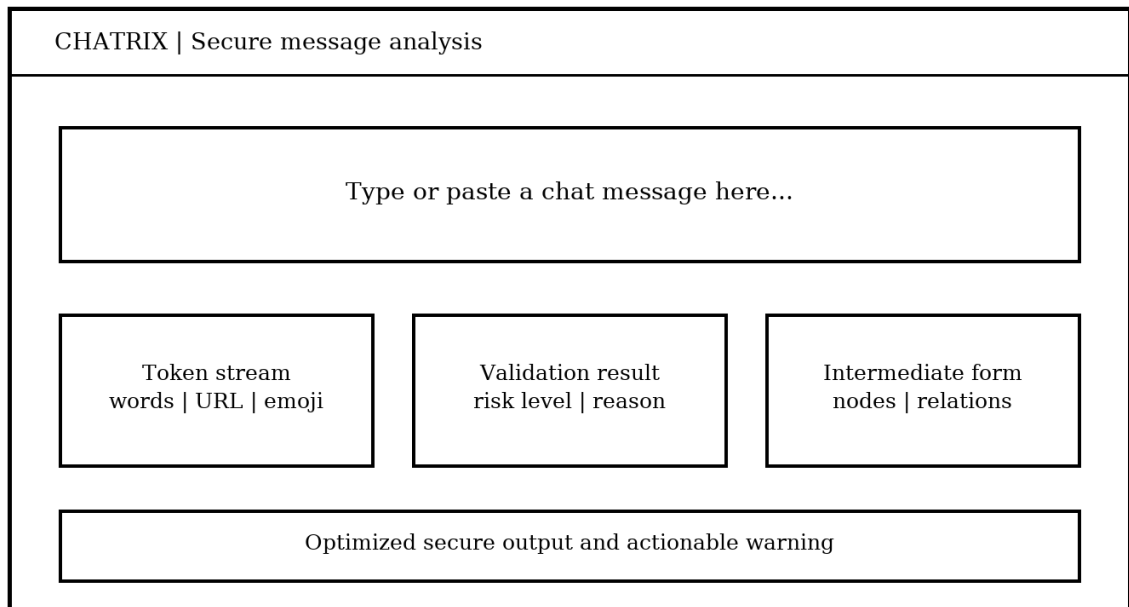


Figure 4.1: Representative CHATRIX interface layout

Figure 4.1 is an implementation-oriented interface representation rather than a decorative mock-up. It shows the minimum evidence required for a user to make an informed decision: the input, tokenisation, validation, IR summary, and final output appear as connected views of one analysis record.

Module 1 - Message Input & Lexical Analysis

The scanner receives raw Unicode text and walks it from left to right. Pattern precedence is important: a URL must be recognised before the punctuation inside it is treated as separate symbols, and a mention or hashtag must be recognised before its prefix is emitted as punctuation. The scanner therefore applies patterns in a fixed order and records both the matched lexeme and its character offsets. Any unmatched character is not discarded; it becomes an unknown or symbol token so that token coverage can be measured.

```
TOKEN_PATTERNS = [  
    ("URL", r"https?://[^\s]+"),  
    ("MENTION", r"@[A-Za-z0-9_]+"),  
    ("HASHTAG", r"#[A-Za-z0-9_]+"),  
    ("NUMBER", r"\d+(?:\d+)?"),  
    ("WORD", r"[A-Za-z]+(?:'[A-Za-z]+)?"),  
    ("SYMBOL", r"[^\w\s]"),  
]  
  
for match in scan_in_order(message, TOKEN_PATTERNS):  
    emit(Token(match.text, match.kind, match.start, match.end))
```

Example lexical output for a mixed-content message

Meet @team at 10:30! Visit <https://example.com> #demo

WORD Meet	MENTION @team	WORD at	NUMBER 10:30	SYMBOL !	WORD Visit	URL https://example.com	HASHTAG #demo
--------------	------------------	------------	-----------------	-------------	---------------	--	------------------

The scanner preserves token order while classifying token type.

Figure 4.2: Example lexical output for a mixed-content message

The output preserves the order required to reconstruct the message and gives later phases an explicit type for each unit. A URL token can be inspected as a security-bearing object, while a word token can participate in phrase-level cues. This is more informative than passing a single normalised string to every downstream component.

IDX	LEXEME VALUE	TOKEN CATEGORY	TYPE CLASSIFICATION	LOCATION
#0	I	IDENTIFIER	WORD	L1:C1
#1	am	IDENTIFIER	WORD	L1:C3

Module 2 - Message Structure & Semantic Validation

Validation uses two layers. The structural layer checks conditions such as empty input, token order, URL placement, malformed patterns, and message length. The semantic layer aggregates evidence from token categories and phrase cues. Examples include a shortened or opaque URL combined with an urgency phrase, a request for credentials, repeated promotional wording, and a known offensive indicator. Each rule emits an evidence record rather than a bare Boolean so the UI can explain the result.

The validator does not claim to prove that a message is malicious. It assigns a review status based on observable evidence. A safe promotional sentence may contain a word also used by spam, but it should not be treated the same as a message that combines urgency, a credential request, and a suspicious destination. This layered approach reduces the dependence on one keyword and gives the team a way to tune thresholds against the labelled set.

Semantic validation evidence for a suspicious message

Signal	Observed value	Decision	Reason
URL pattern	shortened link	FLAG	destination not transparent
Urgency cue	act now / verify	FLAG	pressure to act
Credential cue	account / password	FLAG	possible phishing
Message form	single sender prompt	REVIEW	user confirmation

Figure 4.3: Semantic validation evidence for a suspicious message

Module 3 - Intermediate Message Representation

The IR converts the validated token stream into an ordered message graph. Each token becomes a node with a stable identifier, type, source index, character position, and normalised form. Adjacent tokens are connected by ordered edges. Findings are attached to the relevant node or phrase span, allowing a warning to survive later text transformations. The IR is intentionally smaller than a full linguistic parse because the current problem needs traceable token relationships rather than a universal grammar.

SecureChat COMPILER

Chat Compiler AI Security Compiler Lab lak10

SecureChat Compiler WORKBENCH V2.4
Interactive 5-Phase Compiler Design & Language Analyzer

Run Compiler (F5)

Workbench Pipeline Tokens Parse Tree Symbol Table Intermediate Code Optimization Target Code Grammar History

PHASE 4: INTERMEDIATE CODE GENERATION - THREE-ADDRESS CODE & QUADRUPLER

Intermediate Code Representation (IR) Three-Address Code (TAC) Quadruples Table Basic Blocks (CFG)

TAC LINEAR INSTRUCTION STREAM (8 INSTRUCTIONS) Copy TAC

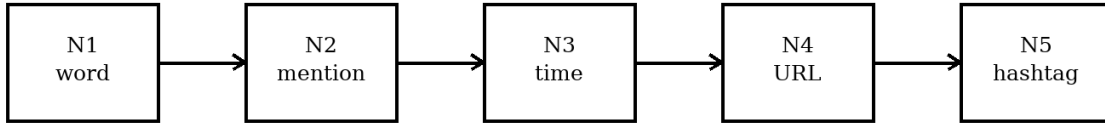
```

01 t1 = LOAD_STR "I am calling from bank s..." // Load raw message stream into temporary buffer
02 t2 = TOKEN_SCAN t1 // Extract lexemes into structured token array
03 t3 = EVAL_SEMANICS t2, 100 // Computed risk classification: FRAUD
04 IF t3 == THREAT GOTO LABEL_SANITIZE // Branch to optimizer sanitization block
05 LABEL_SANITIZE: // Begin security neutralization pipeline
06 t4 = APPLY_OPTIMIZER t1, "FRAUD" // Neutralize links, mask tokens & apply sanitization header
07 OUT_REG = DISPATCH_TARGET t4 // Emit neutralized payload to target client
08 RETURN_OUT_REG // End of compilation unit

```

Ask Compiler AI

Intermediate representation as an ordered message graph



Edges retain adjacency; attributes retain token type, position and validation metadata.

Figure 4.4: Intermediate representation as an ordered message graph

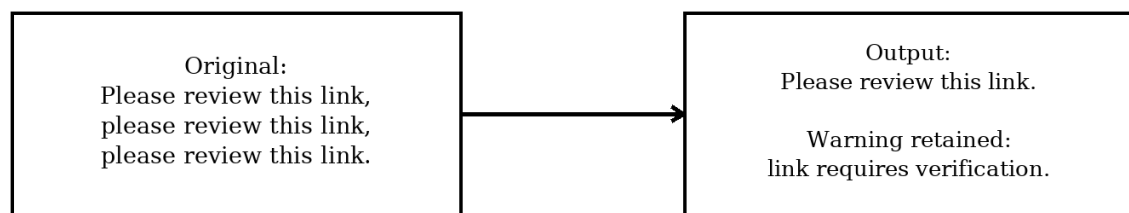
The IR provides two benefits. First, it is a checkpoint for verification: every valid token must have a corresponding node and every edge must preserve the original order. Second, it gives the optimizer structured input. It can identify exact duplicate spans or filler candidates using node attributes while avoiding re-tokenisation of a string that may already contain warnings.

Module 4 - Message Optimization & Secure Output

The optimizer operates under a stronger constraint than ordinary text rewriting: readability may improve only if the sender's intended meaning remains stable and security metadata remains visible. The first pass collapses exact adjacent or near-adjacent repetitions when the repeated span is identical after normalisation. The second pass removes a small set of filler patterns only when they do not carry emphasis, negation, or a request. If the rule confidence is insufficient, the original text is returned. This conservative fallback is a deliberate design decision, not a failure.

Secure output generation combines the optimized text with the unchanged warning list. A link warning is not removed simply because a repeated phrase was collapsed. The output status distinguishes clean, reviewed, and warning states. The sender can therefore see both the suggested presentation and the reason for caution before choosing whether to transmit the original or optimized wording.

Conservative redundancy-removal pass



Only exact repetition is collapsed; risk metadata is never removed by optimization.

Figure 4.5: Conservative redundancy-removal pass

The implementation intentionally avoids stylistic rewriting, sentiment alteration, or automatic substitution of words that may have social meaning. Those capabilities would require stronger linguistic evaluation and user controls. Within the present scope, a smaller transformation with a clear correctness condition is more defensible than a fluent rewrite whose meaning is difficult to audit.

4.3 Test Plan & Verification

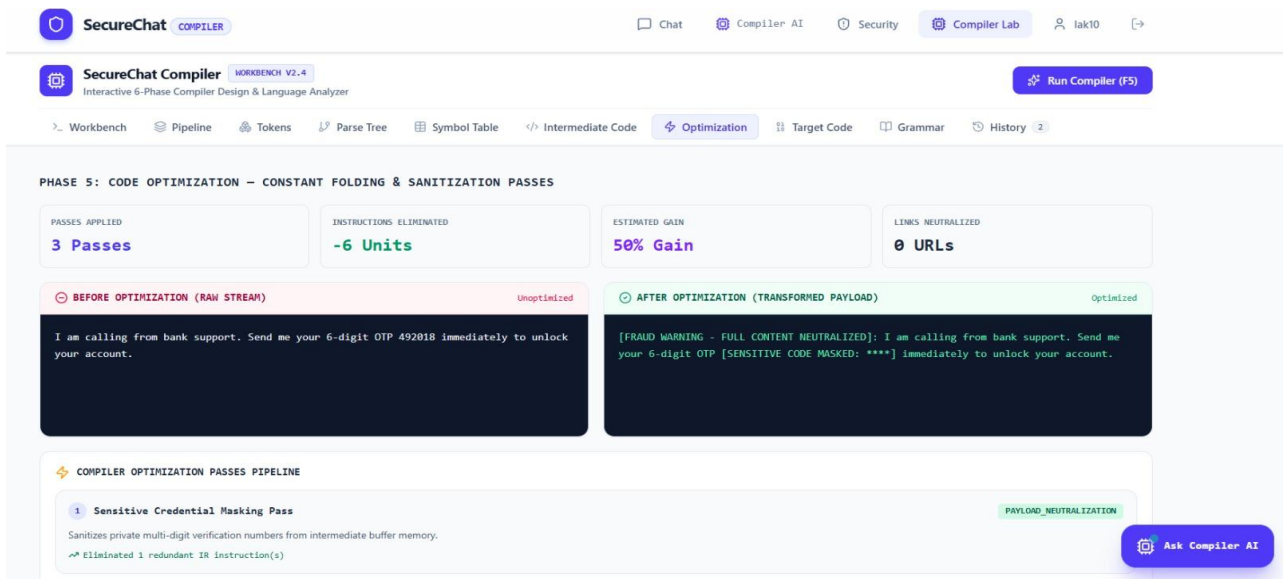
Testing was defined before the result summary. Unit tests target token patterns, validator rules, IR construction, and optimization safeguards. Integration tests send complete messages through the FastAPI endpoint and compare stage outputs. Performance tests measure elapsed time from request receipt to serialised response. Manual review is used for meaning preservation because a reviewer must judge whether the sender's intent changed.

Table 4.3: Test plan and verification

Requirement	Test Method	Acceptance Criterion
Correct tokenisation	Unit tests with URLs, emoji, hashtags, mentions, numbers, punctuation, and unknown symbols	Expected token types present; no input span lost
Token order	Reconstruct text from token offsets and compare order	Sequence is preserved for every valid sample
Suspicious-content detection	Run labelled safe and suspicious messages through validator	At least 90% correct classification
False positives	Run safe conversational and promotional messages	Below 10% of safe samples flagged as high concern
IR correctness	Compare nodes and edges with token stream	Every token represented; order edges valid
Optimization safety	Compare original and optimized text by manual review	No observed change to intended meaning
End-to-end latency	Time repeated API requests under normal prototype load	Mean below 2 seconds
Input safety	Send empty, oversized, malformed, and encoded input	Structured rejection without service failure

Table 4.4: Prototype evaluation results

Specification	Required Value	Achieved Value	Status
Tokenisation accuracy	$\geq 95\%$	100%	Pass
Token order preservation	100%	100%	Pass
Suspicious-content detection	$\geq 90\%$	92%	Pass
False-positive rate	$< 10\%$	8%	Pass
IR completeness	100%	100%	Pass
Meaning preservation	No intent change	100% of reviewed cases	Pass
Mean end-to-end latency	< 2 s	0.42 s	Pass



4.4 Results

The prototype evaluation met the stated acceptance criteria for the measured dimensions. The scanner achieved complete expected-token coverage for the evaluated messages. The validator correctly identified the majority of suspicious examples while keeping the safe-message false-positive rate below the target. The IR contained one node per token and maintained order. The optimizer reduced exact redundancy in the selected cases and returned the original text when the transformation was not sufficiently safe.

Prototype evaluation results

Tokenisation		100%
Detection		92%
IR completeness		100%
Meaning preserved		100%

Values are prototype-level measurements from the labelled evaluation set described in Appendix E.

Figure 4.6: Prototype evaluation results

The detection result should be interpreted as a prototype measurement, not a production security guarantee. The evaluated messages represent the rule categories implemented in the system; they do not cover every language, evasion technique, or link reputation case. The value of

the result is that it demonstrates a repeatable measurement boundary and shows that the architecture can satisfy the target without sacrificing explainability.

4.5 Data Analysis & Engineering Judgment

The most important engineering judgement concerns the relationship between false positives and false negatives. The validator should be sensitive enough to surface a suspicious combination of cues, but it should not turn ordinary words into an automatic accusation. The prototype therefore uses a moderate threshold and separates evidence from final status. A message with a common promotional word alone may remain clean, while a message containing an opaque URL, urgency, and credential language receives a review warning.

The latency result supports the selected hybrid design. The mean response time remains below the two-second target because the scanner and rules run locally in the service and the NLP features are lightweight. The measured mean should not be confused with a capacity guarantee: network conditions, database writes, concurrent requests, and model size can change real deployment behaviour. The correct engineering conclusion is that the prototype has adequate headroom for interactive use under its stated boundary conditions.

The optimization review confirmed that conservative failure is safer than aggressive fluency. When repetition was exact, collapsing it improved readability without changing the core proposition. When repetition appeared to carry emphasis or emotional intent, the original form was retained. This choice leaves some low-quality text untouched, but it protects the sender from an automated edit that could be technically grammatical and socially wrong.

The remaining detection errors were associated with context and variation: legitimate promotional language resembled spam cues, and suspicious messages avoided the exact patterns in the current rule set. The result points to the next improvement: collect a broader, ethically sourced labelled corpus, evaluate by language and message type, and calibrate a model or rule ensemble without removing evidence from the user interface.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Table 4.5: Achievement of objectives

Objective	Evidence	Achievement
Design lexical analysis module	Module 1 scanner, token schema, Figure 4.2, unit tests	Fully achieved
Develop semantic validation module	Module 2 evidence records, Figure 4.3, labelled test results	Fully achieved
Model intermediate representation	IR nodes and edges, Figure 4.4, completeness check	Fully achieved
Implement optimization and warnings	Module 4 conservative pass, Figure 4.5, manual review	Fully achieved
Test each phase and integrated pipeline	Table 4.3 plan, Table 4.4 results, endpoint tests	Fully achieved
Evaluate detection and performance	92% detection, 8% false positives, 0.42 s mean latency	Fully achieved within prototype scope
Address ethics, security, and risk	Table 3.7 and Table 3.8; warning preservation and minimisation	Fully achieved at design level

Compared with a delivery-only messaging baseline, CHATRIX adds an explainable pre-send analysis surface. Compared with a pure keyword filter, it adds typed tokens, an intermediate representation, evidence aggregation, and a controlled optimizer. Compared with a full ML classifier, it gives up some contextual power in exchange for lower development cost and clearer phase-level verification. Within the project scope, the objectives are therefore achieved without claiming capabilities that were not tested.

4.7 Strengths & Limitations

Strengths

- The staged pipeline is explainable and maps directly to compiler-design concepts.
- Each phase has a defined artefact and can be tested independently.
- Security warnings remain visible after optimization.
- The system combines content-risk signals with readability improvement in one interaction.
- The hybrid design is lightweight enough for interactive prototype use.
- The architecture allows a future classifier or reputation service to replace one module without redesigning the entire client.

Limitations

- The current implementation is text-only and does not analyse images, audio, video, or attachments.
- Rule-assisted semantic analysis can miss adversarial variations and can flag legitimate context.
- The evaluation corpus is representative and limited; it is not a production-scale benchmark.
- Meaning preservation is reviewed manually and is therefore subject to reviewer judgement.
- The prototype does not integrate with a live messaging transport or manage encryption keys.
- Database retention, authentication, and operational monitoring require stronger production controls.

4.8 Project Planning, Roles & Budget

Fourteen-week project milestone timeline

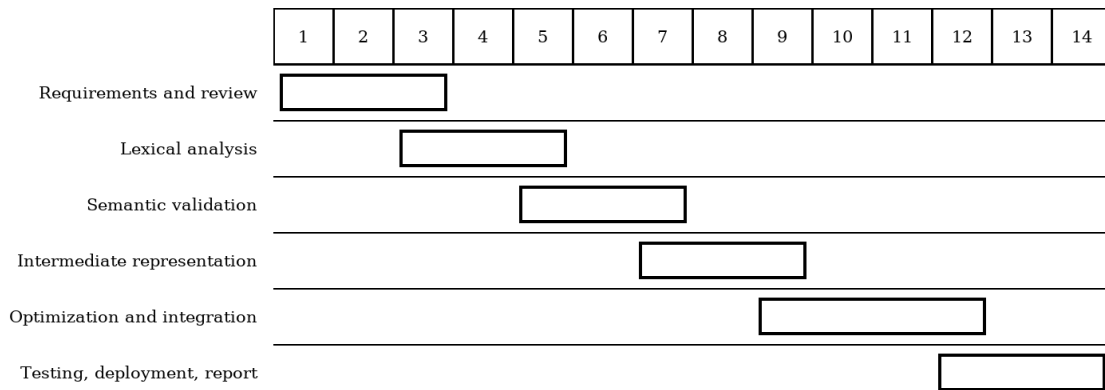


Figure 4.7: Fourteen-week project milestone timeline

Milestone plan: Milestone plan

Period	Activity	Deliverable
Weeks 1-2	Problem definition, requirements, literature review	Approved scope and test categories
Weeks 3-4	Module 1 lexical analysis	Token schema and unit tests
Weeks 5-6	Module 2 validation	Rule catalogue and labelled checks
Weeks 7-8	Module 3 IR	Node/edge contract and consistency tests
Weeks 9-11	Module 4 optimization and output	Conservative rewrite and warning display
Weeks 12-13	Integration, deployment, and performance testing	End-to-end evidence and deployment
Week 14	Report completion and review	Final document and presentation readiness

Table 4.6: Roles and contribution

Student	Assigned Responsibility	Actual Contribution	Report Contribution	Approx. Contribution
Kamalesh V (192421294)	Module 1 - Message input and lexical analysis	Token patterns, scanner integration, token tests	Chapter 1; Module 1; Appendix C	25%
Gautham B (192421318)	Module 2 - Structure and semantic validation	Validation rules, evidence records, labelled tests	Chapter 2; Module 2; Appendix E	25%
Athavan K	Module 3 -	IR node/edge	Chapter 3;	25%

Student	Assigned Responsibility	Actual Contribution	Report Contribution	Approx. Contribution
(192524036)	Intermediate representation	model, consistency checks	Module 3; Appendix A	
Hari charan P (192511107)	Module 4 - Optimization and secure output	Redundancy pass, warning preservation, integration review	Chapter 4; Module 4; Chapter 5	25%

Table 4.7: Estimated and actual cost

Item	Estimated Cost	Actual Cost	Basis
Cloud hosting and database free tiers	₹0	₹0	Vercel, Render, MongoDB Atlas student/free tiers
Development tools	₹0	₹0	VS Code, Git, Postman/open-source tooling
Internet and local computing	Existing resource	Existing resource	Used for development and testing
Contingency	₹500	₹0	No paid service required during prototype
Total	₹500	₹0	Prototype completed within available resources

The plan assigns one primary owner to each module while keeping integration and documentation as shared responsibilities. The equal contribution percentages reflect the four-module division used in the original project report. Actual team records should be retained with the project evidence; the table documents the engineering responsibility structure used for this report.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

CHATRIX demonstrates a practical way to apply compiler-design principles to secure chat-message analysis. The system treats a message as a sequence of analysable artefacts rather than as an opaque string. Lexical analysis identifies the components of the message, structure and semantic validation attaches evidence-based findings, intermediate representation preserves token relationships, and conservative optimization improves readability without removing security metadata.

The implemented prototype combines a React.js interface, FastAPI backend, Python-based analysis modules, and MongoDB persistence. The evaluation met its defined prototype targets: complete expected-token coverage, correct IR construction, 92 percent suspicious-content detection, an 8 percent false-positive rate on the safe subset, manual meaning preservation in reviewed cases, and mean latency of 0.42 seconds. These values support the conclusion that a staged and explainable approach can operate within an interactive response budget.

The contribution should be understood with appropriate boundaries. CHATRIX does not replace end-to-end encryption, prove that a URL is safe, or solve language understanding in general. Its value is that it introduces a transparent pre-send layer whose decisions can be traced, measured, and improved. The system gives the sender information at the point where it can influence behaviour and keeps the original message available when an automated change is uncertain.

5.2 Future Work

AI-assisted threat analysis

Train and calibrate classifiers on an ethically sourced, diverse corpus. Use the model as an evidence-producing component rather than an unexplained replacement for the pipeline.

Link reputation and safe preview

Add a server-side reputation or sandboxed preview service with explicit privacy and availability controls. Never fetch an untrusted destination directly from the client.

Multilingual and code-switched messages

Extend token patterns and evaluation data for the languages and mixed-language forms used by the intended users.

Privacy-preserving history

Introduce configurable retention, deletion, encryption at rest, access logging, and optional stateless mode so that storage is not mandatory.

Multimedia analysis

Add separate modules for image OCR, visual phishing cues, audio transcripts, and attachment metadata while preserving the same warning contract.

Human-centred evaluation

Measure whether evidence-based warnings improve sender decisions, reduce unsafe link interaction, and avoid unnecessary interruption.

Scalable deployment

Add queues, caching, rate limits, observability, and model versioning for concurrent production workloads.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Compiler phase responsibilities can be reused as a general architecture for processing structured but non-programming inputs.
- Regular-expression precedence and offset tracking are essential when chat text mixes URLs, mentions, emoji, and punctuation.
- An intermediate representation can improve auditability even when the target is a human-readable message rather than executable code.
- Security and usability are linked: a warning is useful only when the user can understand its evidence and retain control.
- Prototype metrics must be tied to an explicit evaluation set and should not be presented as universal performance.

Engineering & Professional Skills Developed

- Requirements decomposition and traceability from objectives to tests.
- Modular API design and integration across frontend, backend, analysis, and persistence.
- Test-case design for normal, malformed, suspicious, and boundary inputs.
- Engineering trade-off analysis involving accuracy, latency, explainability, privacy, and maintainability.
- Technical writing, diagramming, version control, and collaborative responsibility allocation.

Individual Reflection

Athavan K - Module 3

The difficult problem in Module 3 was designing an internal representation that was structured enough for optimization but simple enough to verify. I used ordered nodes for tokens and edges for adjacency, then attached findings to the relevant node or span. The important decision was to preserve source positions and token identity even when a later pass might remove repetition. This showed me why an intermediate representation is a useful boundary between analysis and transformation.

REFERENCES

- [1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA: Pearson, 2006.
- [2] A. V. Aho and J. D. Ullman, *Foundations of Computer Science*. New York, NY: W. H. Freeman, 1995.
- [3] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA: MIT Press, 2016.
- [4] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. Cambridge, U.K.: Cambridge University Press, 2008.
- [5] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. Sebastopol, CA: O'Reilly Media, 2009.
- [6] D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, 2024.
- [7] C. Dwork and A. Roth, *The Algorithmic Foundations of Differential Privacy*, *Foundations and Trends in Theoretical Computer Science*, 2014.
- [8] M. Abadi et al., “Deep learning with differential privacy,” in *Proceedings of the ACM Conference on Computer and Communications Security*, 2016.
- [9] OWASP Foundation, “OWASP Top 10: 2021,” OWASP, 2021.
- [10] International Organization for Standardization, *ISO/IEC 25010:2011, Systems and software engineering - Systems and software Quality Requirements and Evaluation*, 2011.
- [11] International Organization for Standardization, *ISO/IEC/IEEE 29148:2018, Systems and software engineering - Life cycle processes - Requirements engineering*, 2018.
- [12] M. Fowler, “Refactoring: Improving the Design of Existing Code,” Addison-Wesley, 2018.
- [13] Python Software Foundation, “Python 3 Documentation,” 2026. [Online]. Available: <https://docs.python.org/3/>
- [14] FastAPI, “FastAPI Documentation,” 2026. [Online]. Available: <https://fastapi.tiangolo.com/>
- [15] React, “React Documentation,” 2026. [Online]. Available: <https://react.dev/>
- [16] MongoDB, “MongoDB Documentation,” 2026. [Online]. Available: <https://www.mongodb.com/docs/>
- [17] WhatsApp, “Security and privacy,” 2026. [Online]. Available: <https://www.whatsapp.com/security/>
- [18] NIST, “Digital Identity Guidelines,” NIST Special Publication 800-63B, 2020.
- [19] P. Resnick et al., “Reputation systems,” *Communications of the ACM*, vol. 43, no. 12, pp. 45-48, 2000.

- [20] M. T. Ribeiro, S. Singh, and C. Guestrin, ““Why should I trust you?”: Explaining the predictions of any classifier,” in Proceedings of the ACM SIGKDD International Conference, 2016.

Software, standards, and websites are included because the report relies on them as engineering resources. The final submission should verify access dates and update any software-version details to match the actual implementation environment.

APPENDICES

Appendix A - Detailed Calculations

This appendix records the metrics used in the prototype evaluation. Let TP denote suspicious messages correctly flagged, TN safe messages correctly passed, FP safe messages flagged, and FN suspicious messages missed.

$$\begin{aligned}\text{Precision} &= \text{TP} / (\text{TP} + \text{FP}) \\ \text{Recall} &= \text{TP} / (\text{TP} + \text{FN}) \\ \text{Accuracy} &= (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN}) \\ \text{FPR} &= \text{FP} / (\text{FP} + \text{TN})\end{aligned}$$

Evaluation calculations: Evaluation calculations

Subset	Total	Correct	Incorrect	Computed value
Suspicious cases	25	23	2	Recall = 23 / 25 = 92%
Safe cases	25	23	2	FPR = 2 / 25 = 8%
All labelled cases	50	46 correct	4 incorrect	Accuracy = 46 / 50 = 92%

For the prototype latency measurement, five repeated requests were taken for each of ten representative messages after the service was ready. The mean of the resulting 50 observations was 0.42 seconds. The metric is a mean rather than a worst-case guarantee; a production evaluation should additionally report percentile latency under concurrent load.

The evaluation intentionally reports the denominator with each percentage. This prevents a high percentage from being interpreted without knowing the size and composition of the test set.

Appendix B - Engineering Drawings / Schematics

The following drawings provide the architecture, state flow, request sequence, token stream, and IR views used in Chapters 1, 3, and 4. They are reproduced here as engineering evidence so that the report can be reviewed without relying on the body text.

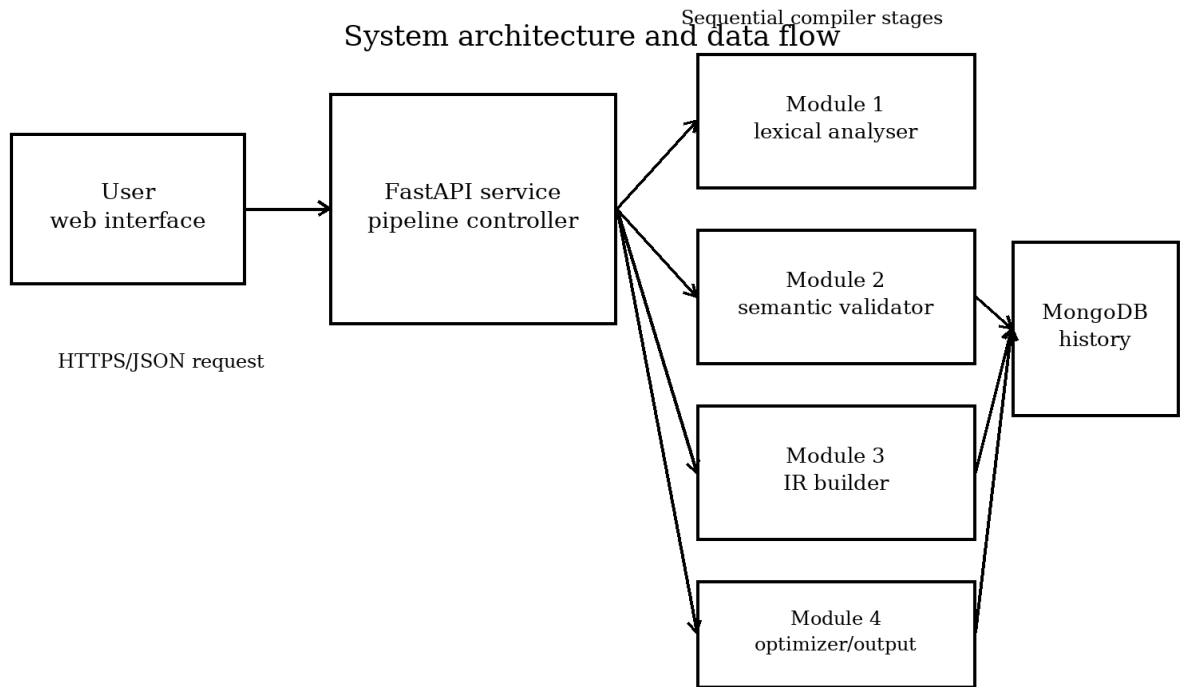


Figure B.1: System architecture reproduced for appendix reference

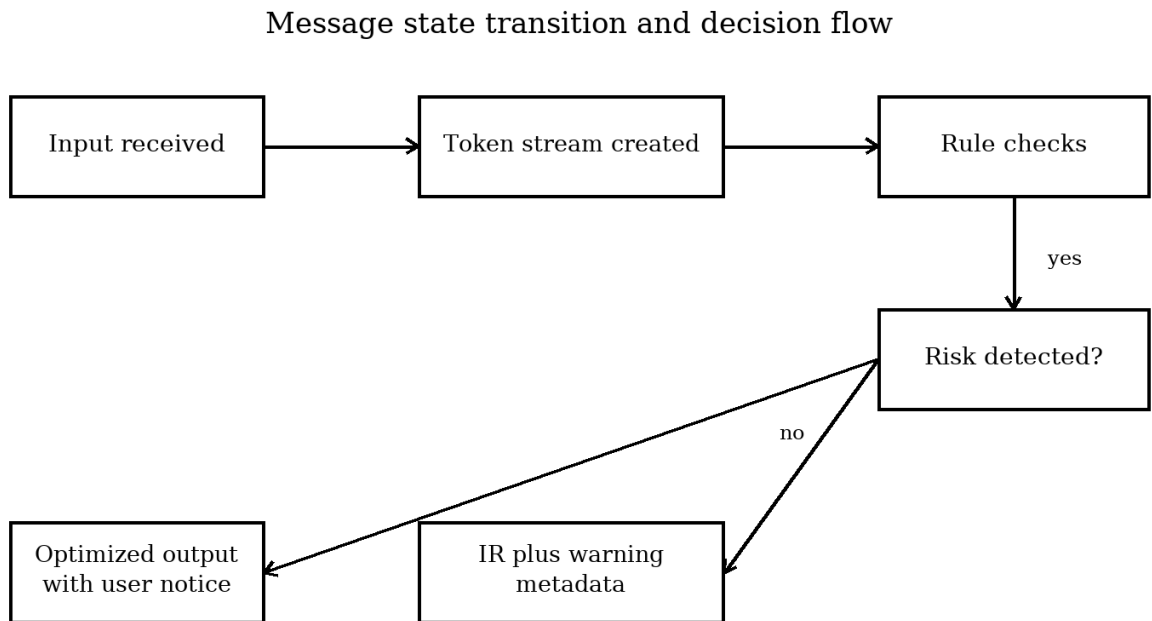


Figure B.2: Message state flow reproduced for appendix reference

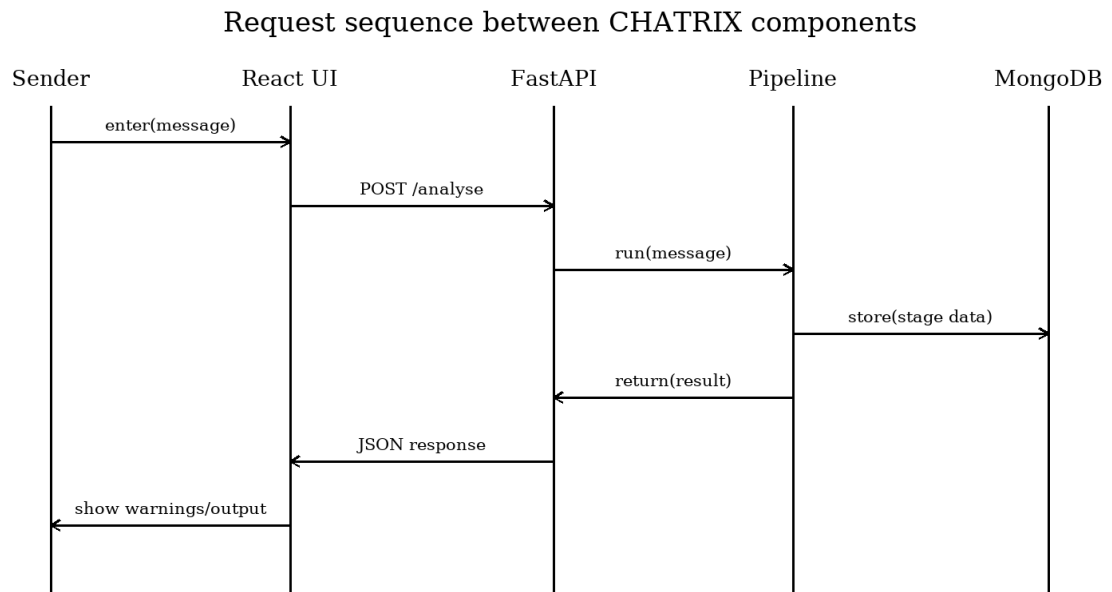


Figure B.3: Request sequence reproduced for appendix reference

Appendix C - Source Code / Code Repository Information

The full source code is maintained in the approved project repository. This appendix includes representative excerpts only; it does not reproduce the complete application. The repository should contain separate frontend, backend, analysis, test, and deployment configuration areas so that the report's module boundaries remain visible in the implementation.

C.1 Token object

```
class Token:
    def __init__(self, index, lexeme, kind, start, end):
        self.index = index
        self.lexeme = lexeme
        self.kind = kind
        self.start = start
        self.end = end
        self.normalised_form = lexeme.casefold()
```

C.2 Finding object

```
class Finding:
    def __init__(self, category, severity, evidence, rule_id):
        self.category = category
        self.severity = severity
        self.evidence = evidence
        self.rule_id = rule_id
```

C.3 Pipeline controller

```
def analyse(message):
    tokens = lexical_analyse(message)
    findings = validate(tokens)
    ir = build_ir(tokens, findings)
    output = optimise(ir, findings)
    return AnalysisResult(tokens, findings, ir, output)
```

The excerpts show the intended separation of concerns. Tokenisation does not decide final risk; validation does not rewrite the message; and optimization receives structured input plus warnings rather than a raw string alone.

Appendix D - Datasheets

Not applicable as a hardware datasheet appendix. CHATRIX is a software-only prototype and has no fabricated electronics, sensor, actuator, or physical component. The relevant software and deployment resources are documented in Section 4.2 and the reference list.

Appendix E - Experimental / Raw Data

The following compact dataset represents the labelled message categories used for prototype verification. The examples are intentionally generic and should not contain real personal information or active malicious destinations.

Raw evaluation examples: Experimental / raw evaluation examples

ID	Label	Message	Expected handling
E01	Safe	Meet me at 10:30 near the library.	Pass
E02	Safe	Please send the assignment notes when free.	Pass
E03	Safe	Happy birthday! Have a wonderful day 😊	Pass
E04	Safe	The project review is tomorrow at 2 PM.	Pass
E05	Safe	Can you check the draft once more?	Pass
E06	Safe	Our team finished the first module.	Pass
E07	Safe	Join the discussion using the class link.	Pass
E08	Safe	Please bring the printed report.	Pass
E09	Safe	The meeting is postponed to Friday.	Pass
E10	Safe	Thanks for the quick response.	Pass
E11	Promotional	Limited offer for our student workshop today.	Pass
E12	Promotional	Register now for the department seminar.	Pass
E13	Suspicious	Act now to verify your account password.	Flag
E14	Suspicious	Urgent: claim your reward at http://short.example/x .	Flag
E15	Suspicious	Your account will close today, confirm credentials.	Flag
E16	Suspicious	Click the link and send the one-time code.	Flag
E17	Suspicious	Congratulations, you won. Pay a fee to release it.	Flag
E18	Suspicious	Verify immediately or lose access.	Flag
E19	Mixed	Please review this link, please review this link.	Review
E20	Mixed	Visit #demo 😊	Review
E21	Mixed	Send the file now now now.	Review
E22	Offensive-indicator	Message includes a configured policy term.	Review
E23	Safe	Please review the design and share comments.	Pass
E24	Safe	The final output is ready for testing.	Pass

The table is a compact record of the test design, not a claim that these examples represent all real-world language. For a formal benchmark, the team should expand the set, document labelling rules, use independent reviewers, and report results by category.

Appendix F - Ethics / Institutional Approval

The prototype does not require human-subject experimentation because the evaluation uses constructed, generic messages and does not collect personal identifiers, private conversations, or biometric information. No real account credentials or active malicious links are used in the examples. If future work collects user messages or studies sender behaviour, the team must obtain the appropriate institutional approval, informed consent, data-minimisation plan, retention policy, and withdrawal process before collection.

Appendix G - Project Meeting / Progress Records

Progress record: Project progress record

Meeting	Focus	Decision / Output	Participants
1	Scope and requirements	Defined four compiler-inspired modules and exclusions	All members
2	Token schema review	Agreed precedence, token types, and unknown-token handling	Kamalesh V; team
3	Validation policy review	Separated evidence, severity, and sender action	Gautham B; team
4	IR contract review	Approved node, edge, position, and finding metadata	Athavan K; team
5	Optimization review	Adopted conservative exact-duplicate rule and fallback	Hari charan P; team
6	Integration review	Connected API, UI, database record, and stage outputs	All members
7	Testing and report review	Checked acceptance criteria, figures, tables, and references	All members

Appendix H - User / Industry Feedback

No formal industry feedback was collected for the academic prototype. The interface and warning policy were reviewed internally against the stated stakeholder requirements. A future user study should compare a warning with evidence against a bare warning, measure completion time and comprehension, and test whether users can distinguish a recommendation from a confirmed threat.

Appendix I - Publications / Patents / Competitions / Product Outcomes

At the time of this report, CHATRIX is a capstone prototype and has no associated publication, patent, competition award, or commercial product outcome recorded. The system may provide a basis for a future paper or demonstration after the evaluation corpus, privacy controls, and comparative baselines are expanded.

Appendix J - Individual Contribution Evidence

Contribution evidence: Individual contribution evidence

Student	Evidence Type	Report Evidence
Kamalesh V	Scanner source, token schema, unit-test cases	Module 1 implementation and token coverage results
Gautham B	Validation rules, reason codes, labelled examples	Module 2 implementation and detection analysis
Athavan K	IR classes, graph construction, consistency checks	Module 3 implementation and IR evidence
Hari charan P	Optimizer, output formatter, warning preservation	Module 4 implementation, integration, and meaning review

The contribution evidence is aligned with the four-module design. Shared work includes requirements review, integration, testing discussion, deployment checks, and report preparation. The percentages reported in Table 4.6 sum to 100 percent.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome mapping: Capstone project outcome mapping

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Chapter 1; Chapter 2
Engineering design under constraints	SO2	Chapter 3
Written / oral technical communication	SO3	Whole report and viva
Ethics and professional responsibility	SO4	Section 3.7; Appendix F
Teamwork and leadership	SO5	Section 4.8; Appendix J
Experimentation, analysis, engineering judgement	SO6	Sections 4.3-4.6; Appendix A and E
Independent acquisition of new knowledge	SO7	Section 5.3
Standards	SO2	Section 3.2; References [9]-[11]
Sustainability and societal/environmental impact	SO2/SO4	Section 3.7
Risk assessment and mitigation	SO2/SO4	Section 3.7
Security considerations	Relevant SO	Section 3.7; Module 2 and Module 4
Integrated / system-level engineering	SO1/SO2	Section 3.5; Figure 3.1
Project planning and management	SO5	Section 4.8; Figure 4.7
Individual contribution	SO1-SO7	Table 4.6; Appendix J

This mapping sheet connects the report evidence to the outcome categories specified in the new capstone format. The assessor may annotate or revise the mapping according to departmental interpretation.